# SDR Spectrum Monitoring Sensor

Technical Resume

Repository-grounded summary from source code and Diagrama-ANE

February 9, 2026

---

## 1. Executive Summary

The system is a sensor-side SDR platform that captures radio spectrum data, uploads measurements to a cloud API, reports device health, performs optional frequency calibration, and supports low-latency demodulated audio streaming.

At runtime, the architecture is split into:

- **RF data plane (C):** `rf_app` controls HackRF, computes PSD, and handles demodulated audio.
- **Control plane (Python):** `orchestrator.py` manages realtime commands and scheduled campaigns.
- **Reliability and telemetry:** `campaign_runner.py`, `retry_queue.py`, `status.py`.
- **Network/GPS subsystem (C):** `ltegps_app` handles LTE/GPS posting.
- **Streaming bridge:** `server_webrtc.py` translates local Opus/TCP frames to WebRTC.

## 2. Main Components

### Python Orchestration

- `orchestrator.py`: polls realtime and campaign endpoints, drives acquisition loops, controls realtime demod/streaming.
- `functions.py`: state machine (`IDLE/CAMPAIGN/REALTIME/KALIBRATING`), campaign cron sync, dual-acquisition correction (`AcquireDual`).
- `campaign_runner.py`: one-shot campaign execution, upload, local queue/historic persistence.
- `retry_queue.py`: ordered resend of locally queued JSON payloads.
- `status.py` + `utils/status_util.py`: system metrics snapshot and status upload.

### C Binaries

- `rf/rf.c`: HackRF lifecycle, command parsing, PSD generation (Welch/PFB), optional FM/AM demod, ZMQ I/O.
- `gps-lte/gps-lte.c`: LTE PPP management, GPS read/parse, periodic `/gps` post.
- `common/bacn_gpio.c`: LTE power/reset and antenna selector GPIO control.

## 3. End-to-End Workflow

### Realtime mode

1. Orchestrator requests realtime config from cloud.
2. Config is sent over local ZMQ IPC (`ipc:///tmp/rf_engine`) to `rf_app`.
3. `rf_app` tunes hardware, acquires IQ, computes PSD, and returns JSON results over ZMQ.
4. Orchestrator formats and uploads payload to `/data`.
5. If demodulation is enabled, orchestrator starts `server_webrtc.py`; C audio thread sends Opus frames to local TCP 9000.

### Campaign mode

1. Orchestrator polls campaigns and selects one active campaign.
2. Scheduler writes RF params to shared memory (`/dev/shm/persistent.json`) and upserts a cron job.
3. `campaign_runner.py` executes acquisition and attempts upload.
4. Failed uploads are stored in `Queue/`; successful uploads can be archived in `Historic/`.
5. `retry_queue.py` replays queued files oldest-first.

### Telemetry and GPS

- `status.py` periodically posts system metrics (`/status`).
- `ltegps_app` posts GPS telemetry to `API_URL/gps`.

## 4. Interfaces and Data Contracts

- **HTTP GET:** `/{mac}/realtime`, `/{mac}/campaigns`.
- **HTTP POST:** `/data`, `/status`, and C-side `/gps`.
- **Local IPC:** ZMQ PAIR JSON command/result exchange on `ipc:///tmp/rf_engine`.
- **Audio framing:** custom Opus header (`!IIIHH`) over TCP 9000 before WebRTC publish.
- **Shared state:** JSON key/value store in `/dev/shm/persistent.json`.

## 5. Deployment Model

- CMake builds `rf_app` and `ltegps_app`.
- `install.sh` provisions dependencies, virtual environment, binaries, and services.
- `init_sys.py` generates systemd units/timers and resets cron/shared-memory state.
- Timers execute `status.py` and `retry_queue.py` periodically.

## 6. Diagram-to-Code Alignment (Diagrama-ANE V2)

| Diagram block | Code mapping | Implementation status |
|---|---|---|
| Orchestrator | `orchestrator.py` | Implemented |
| Realtime Superloop | `run_realtime_logic()` | Implemented |
| campaign_runner | `campaign_runner.py` | Implemented |
| kal_sync | `kal_sync.py` | Implemented (not invoked in orchestrator currently) |
| SDR | `rf/rf.c` | Implemented |
| Mux / Antenna ports 1–4 | `common/bacn_gpio.c` | Implemented |
| `/data`, `/status`, `/realtime` | Python API client + orchestrator/status paths | Implemented |
| `/gps` | `gps-lte/libs/utils.c` | Implemented (C-side posting) |
| `/jobs` | No client flow found in repository | Not implemented in this repo |
| Cloud post-processing/auth/storage blocks | Cloud-side conceptual blocks | External / out of scope of this repo |

## 7. Key Risks and Gaps

- Campaign calibration call is present but commented out in orchestrator.
- Diagram request fields (`start_freq_hz/end_freq_hz/resolution_hz`) differ from current runtime schema (`center_freq_hz/sample_rate_hz/rbw_hz`).
- `/jobs` appears in diagram but is not implemented by sensor code.
- No automated test suite is present in the repository.

## 8. Primary Evidence Files

- `orchestrator.py`, `campaign_runner.py`, `status.py`, `retry_queue.py`
- `functions.py`, `utils/request_util.py`, `utils/io_util.py`, `utils/status_util.py`
- `rf/rf.c`, `rf/libs/parser.c`, `rf/libs/psd.c`
- `gps-lte/gps-lte.c`, `gps-lte/libs/utils.c`, `common/bacn_gpio.c`
- `init_sys.py`, `install.sh`, `CMakeLists.txt`, `Diagrama-ANE`